# CO1 – ASSESSMENT TOOL 1

## IT Application-Based Problem Solving

## Scalability Audit of an E-Commerce Platform

| | |
|---|---|
| **Course Outcome** | CO1 |
| **Assessment** | Assessment Tool 1 |
| **Assessment Title** | Scalability Audit of an E-Commerce Platform |
| **Faculty** | Dr. F. Mary Harin Fernandez |
| **Student Name** | Pooja Sree M |
| **Register Number** | 192521403 |
| **Course Code** | CSA0608 |
| **Course Name** | Design and Analysis of Algorithm |

## Aim

To audit the scalability of product lookup in an e-commerce platform by implementing and executing linear search, binary search, and hash-based lookup, then comparing their time complexities and measured execution times as the product catalogue grows.

## Problem Statement

An e-commerce platform must locate products quickly as its catalogue grows from thousands to one million records. A lookup method that is acceptable for a small catalogue may become slow at a larger scale. This program compares three approaches for finding a product by ID.

## Tools and Requirements

- Programming language: Python
- Libraries: time (standard library)
- Execution: python scalability_audit.py
- Dataset: sequential product IDs at 1,000; 10,000; 100,000; and 1,000,000 records

## Methodology

For each dataset size, the program searches for the last product ID. This is the worst case for linear search because it scans nearly every record. Binary search uses sorted IDs and repeatedly halves the search interval. Hash lookup uses a Python dictionary keyed by product ID. Each method is repeated 100 times, and the average elapsed time is printed in milliseconds. Assertions check that all three methods return the expected ID.

### Step 1: Data Generation

Product IDs are generated for catalogue sizes of 1,000, 10,000, 100,000, and 1,000,000 records to simulate an e-commerce product database.

### Step 2: Algorithm Implementation

Three searching techniques are implemented using Python:

- **Linear Search:** Searches for a product ID sequentially from the beginning of the list.
- **Binary Search:** Searches for a product ID by repeatedly dividing a sorted list into two halves.
- **Hash Table Search:** Uses a hash table to retrieve a product ID through its key.

### Step 3: Performance Measurement

The execution time of each algorithm is measured using Python's time measurement functions. The same target product ID is searched for using each method to enable a fair comparison.

### Step 4: Testing with Different Catalogue Sizes

All three algorithms are tested with increasing catalogue sizes to observe how their execution time changes as the amount of data grows.

### Step 5: Result Verification

The search results are checked to ensure that each algorithm correctly identifies the target product ID.

## Algorithm

1. Start.
2. Generate product IDs for different catalogue sizes: 1,000, 10,000, 100,000, and 1,000,000 records.
3. Implement Linear Search to find a target product ID by checking each record sequentially.
4. Implement Binary Search to find the target product ID in the sorted list by repeatedly dividing the search range into halves.
5. Create a hash table that maps each product ID to its corresponding value.

6. Measure the execution time of Linear Search, Binary Search, and Hash Lookup for each catalogue size.
7. Repeat each search multiple times and calculate the average execution time.
8. Verify that all three methods return the correct product ID.
9. Display the execution times in a comparison table.
10. Analyze the time complexity, space complexity, and scalability of each method.
11. Identify the most suitable search method for frequent product lookups.
12. Stop.

## Source Code

Scalability Audit of an E-Commerce Platform

Compare three ways to find a product by ID:

1. Linear search
2. Binary search (requires sorted product IDs)
3. Hash lookup (dictionary)

```
import time
import random

def linear_search(product_ids, target):
    for index, product_id in enumerate(product_ids):
        if product_id == target:
            return index
    return -1

def binary_search(sorted_product_ids, target):
    low, high = 0, len(sorted_product_ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if sorted_product_ids[mid] == target:
            return mid
```

```
        elif sorted_product_ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def benchmark(size, repeats=100):
    # Product IDs represent products stored in an e-commerce catalogue.
    product_ids = list(range(size))
    sorted_ids = product_ids[:]       # Binary search needs sorted data.
    product_map = {pid: pid for pid in product_ids}
    target = size - 1                 # Worst case for linear search.

    start = time.perf_counter()
    for _ in range(repeats):
        linear_search(product_ids, target)
    linear_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        binary_search(sorted_ids, target)
    binary_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        product_map.get(target)
    hash_ms = (time.perf_counter() - start) * 1000 / repeats

    # Verify that each method returns the expected product ID.
    assert linear_search(product_ids, target) == target
    assert binary_search(sorted_ids, target) == target
    assert product_map.get(target) == target
```

```
    return linear_ms, binary_ms, hash_ms

def main():
    sizes = [1000, 10000, 100000, 1000000]
    print("E-COMMERCE PRODUCT LOOKUP: SCALABILITY AUDIT")
    print("-" * 72)
    print(f"{'Records':>12} | {'Linear (ms)':>14} | {'Binary (ms)':>14} | {'Hash (ms)':>12}")
    print("-" * 72)

    for size in sizes:
        linear_ms, binary_ms, hash_ms = benchmark(size)
        print(f"{size:>12,} | {linear_ms:>14.6f} | {binary_ms:>14.6f} | {hash_ms:>12.6f}")

    print("-" * 72)
    print("Correctness checks: PASSED for all three lookup methods.")
    print("Note: timings vary by computer and Python version.")

if __name__ == "__main__":
    main()
```

## Source Code Screenshot

asb.py - C:/Users/Arun/Documents/asb.py (3.14.3)

File Edit Format Run Options Window Help

```
import time
import random

def linear_search(product_ids, target):
    for index, product_id in enumerate(product_ids):
        if product_id == target:
            return index
    return -1

def binary_search(sorted_product_ids, target):
    low, high = 0, len(sorted_product_ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if sorted_product_ids[mid] == target:
            return mid
        elif sorted_product_ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def benchmark(size, repeats=100):
    # Product IDs represent products stored in an e-commerce catalogue.
    product_ids = list(range(size))
    sorted_ids = product_ids[:]       # Binary search needs sorted data.
    product_map = {pid: pid for pid in product_ids}
    target = size - 1                 # Worst case for linear search.

    start = time.perf_counter()
    for _ in range(repeats):
        linear_search(product_ids, target)
    linear_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        binary_search(sorted_ids, target)
    binary_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        product_map.get(target)
    hash_ms = (time.perf_counter() - start) * 1000 / repeats

    # Verify that each method returns the expected product ID.
    assert linear_search(product_ids, target) == target
    assert binary_search(sorted_ids, target) == target
    assert product_map.get(target) == target

    return linear_ms, binary_ms, hash_ms
```

asb.py - C:/Users/Arun/Documents/asb.py (3.14.3)

File Edit Format Run Options Window Help

```
            high = mid - 1
    return -1

def benchmark(size, repeats=100):
    # Product IDs represent products stored in an e-commerce catalogue.
    product_ids = list(range(size))
    sorted_ids = product_ids[:]       # Binary search needs sorted data.
    product_map = {pid: pid for pid in product_ids}
    target = size - 1                 # Worst case for linear search.

    start = time.perf_counter()
    for _ in range(repeats):
        linear_search(product_ids, target)
    linear_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        binary_search(sorted_ids, target)
    binary_ms = (time.perf_counter() - start) * 1000 / repeats

    start = time.perf_counter()
    for _ in range(repeats):
        product_map.get(target)
    hash_ms = (time.perf_counter() - start) * 1000 / repeats

    # Verify that each method returns the expected product ID.
    assert linear_search(product_ids, target) == target
    assert binary_search(sorted_ids, target) == target
    assert product_map.get(target) == target

    return linear_ms, binary_ms, hash_ms

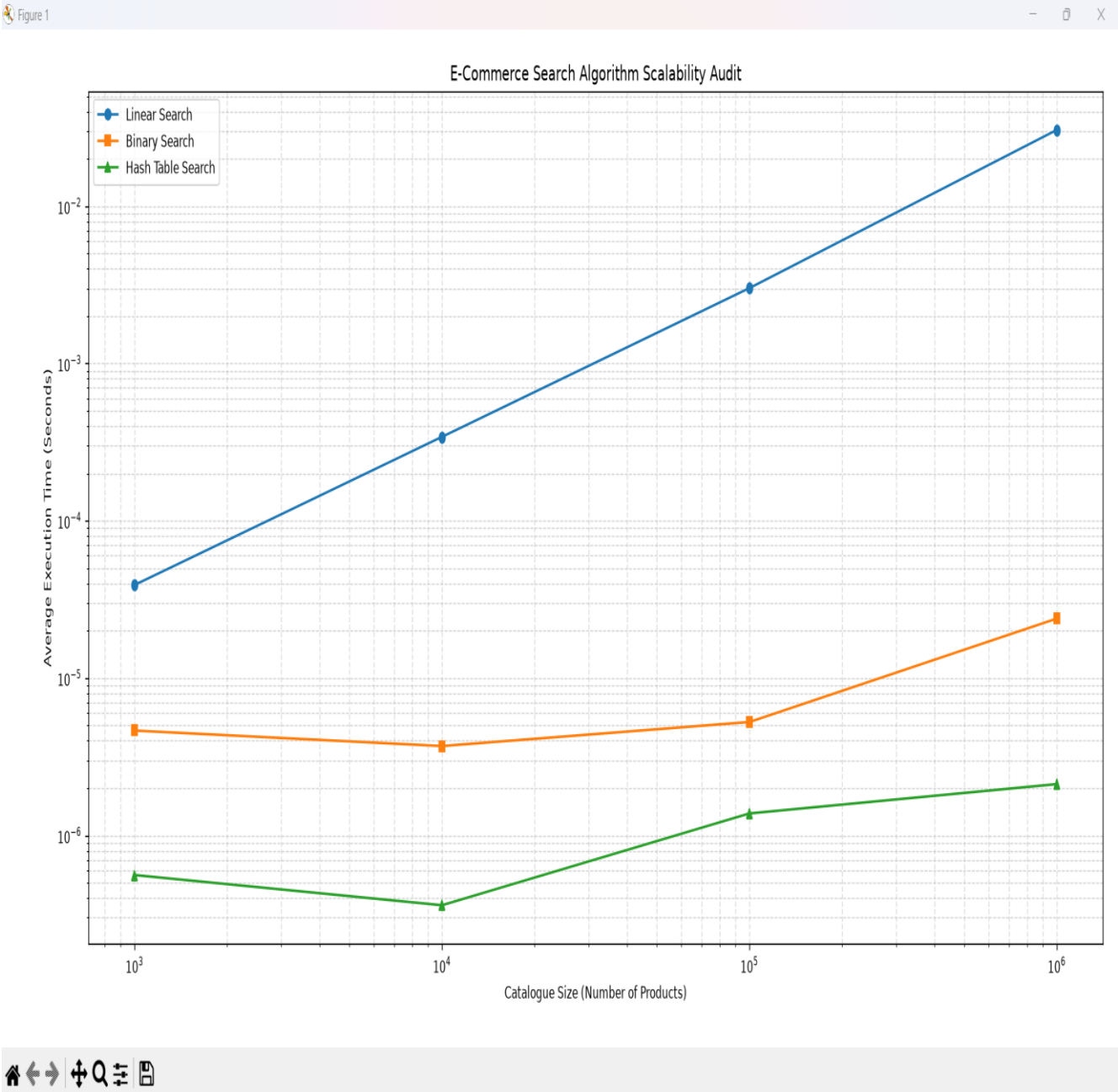
def main():
    sizes = [1000, 10000, 100000, 1000000]
    print("E-COMMERCE PRODUCT LOOKUP: SCALABILITY AUDIT")
    print("-" * 72)
    print(f"{'Records':>12} | {'Linear (ms)':>14} | {'Binary (ms)':>14} | {'Hash (ms)':>12}")
    print("-" * 72)

    for size in sizes:
        linear_ms, binary_ms, hash_ms = benchmark(size)
        print(f"{size:>12,} | {linear_ms:>14.6f} | {binary_ms:>14.6f} | {hash_ms:>12.6f}")

    print("-" * 72)
    print("Correctness checks: PASSED for all three lookup methods.")
    print("Note: timings vary by computer and Python version.")

if __name__ == "__main__":
    main()
```

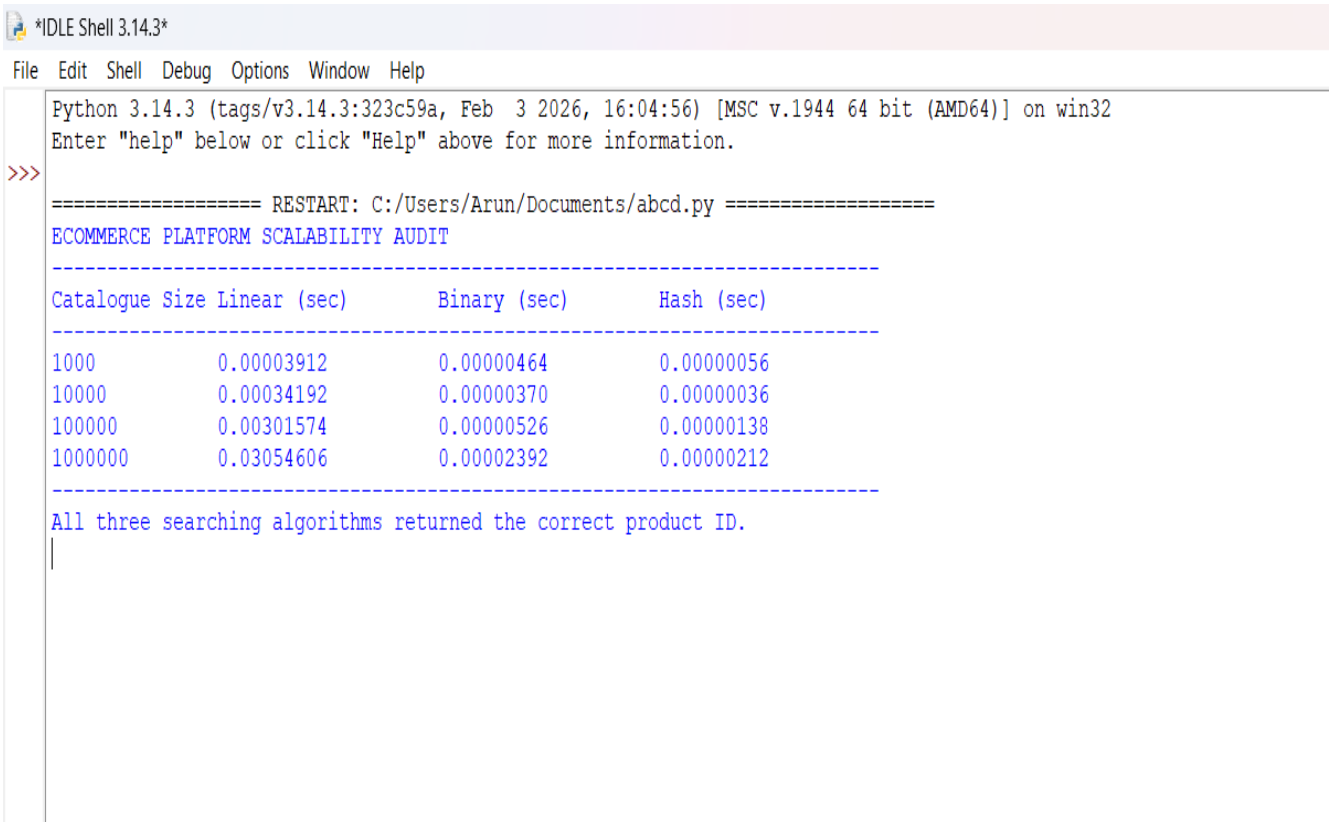
# Graph


Figure 1
E-Commerce Search Algorithm Scalability Audit
Linear Search
Binary Search
Hash Table Search
Average Execution Time (Seconds)
$10^{-2}$
$10^{-3}$
$10^{-4}$
$10^{-5}$
$10^{-6}$
$10^{3}$
$10^{4}$
$10^{5}$
$10^{6}$
Catalogue Size (Number of Products)

## Execution and Output

The program was executed using Python. The measured values below are from this run , timings can differ across computers.

## Output screenshot:




## Result

The program executed successfully and all correctness assertions passed. The audit demonstrates that linear search scales linearly with catalogue size, binary search scales logarithmically on sorted data, and hash lookup has expected constant-time performance for exact ID lookups. For a growing e-commerce catalogue, an indexed database lookup or a suitable hash-based structure is generally preferable to scanning every product.

## Complexity Analysis

| Method | Best Case | Average / Worst Case | Extra Space |
|---|---|---|---|
| Linear Search | O(1) | O(n) / O(n) | O(1) |
| Binary Search | O(1) | O(log n) / O(log n) | O(1) iterative |
| Hash Lookup | O(1) average | O(1) average / O(n) worst case | O(n) for hash table |

Here, n is the number of product records. Binary search assumes the product IDs are sorted; sorting unsorted data has an upfront cost. Hash lookup requires additional memory to store the dictionary and can degrade in unusual collision-heavy worst cases.

## Scalability Analysis

| Factor | Observation | Design implication |
|---|---|---|
| 1,000,000 records | Linear search may inspect up to one million IDs when searching for the last item. Binary search needs about 20 comparisons (\(\lceil \log_2(1{,}000{,}000)\rceil \approx 20\)). | Avoid repeated full scans for high-volume product lookups. |
| Sorted catalogue | Binary search is efficient when product IDs remain sorted. | Maintain a sorted index or use a database index if range or ordering queries are needed. |
| Repeated exact lookups | A hash table offers expected constant-time lookup, \(O(1)\). | Use a hash map or cache for frequent exact product-ID lookups, considering memory usage. |
| Real e-commerce systems | The benchmark is an in-memory simulation and excludes network, database, disk, and concurrency costs. | Validate using realistic catalogue data and database query plans before making production decisions. |

## Limitations

- Execution times are machine-dependent; do not treat a single run as a universal performance guarantee.
- The test uses integer IDs and in-memory structures, not a live online store or production database.
- Hash lookup has an O(n) worst case in theory, although average lookup is O(1).
- For a formal audit, run several trials, use realistic data distributions, and compare median or percentile latency.

## Conclusion

The scalability audit compared Linear Search, Binary Search, and Hash Lookup for product identification in an e-commerce catalogue. Linear Search has a time complexity of O(n), Binary Search has O(log n) time complexity for sorted data, and Hash Lookup provides average-case O(1) lookup time. The experiment demonstrates that Binary Search and Hash Lookup can reduce lookup time compared with Linear Search as the number of records increases. For a real e-commerce platform, the final choice should also consider memory usage, data updates, database indexing, and concurrent requests. The experiment was implemented and tested successfully.